

Exemples pratiques de hooks pour automatiser le workflow

Auto-format après édition

Hook `PostToolUse` sur `Edit|Write`. Détecte l'extension du fichier modifié et applique le formateur adapté.

```
INPUT = $( cat ) FILE = $( echo " $INPUT "
| jq -r '.tool_input.file_path' ) EXT =
" ${FILE} ## * . } " case " $EXT " in
ts | tsx ) npx prettier --write " $FILE
" 2 > /dev/null ;; py ) black "
$FILE " 2 > /dev/null ;; go )
gofmt -w " $FILE " 2 > /dev/null ;;
rs ) rustfmt " $FILE " 2 > /dev/null
;; esac exit 0
```

Configurer en `async: true` — le formatage est cosmétique et ne doit pas bloquer Claude.

Notification macOS à la fin

Hook `Stop`. Déclenche une alerte système macOS avec un son contextuel (succès, erreur, attente).

```
INPUT = $( cat ) MESSAGE = $( echo " $INPUT
" | jq -r '.message // "Terminé"' )
osascript -e "display notification \" $MESSAGE
\" \" with title \" Claude Code \" \" afplay
/System/Library/Sounds/Hero.aiff & exit 0
```

Sécurité : bloquer les commandes dangereuses

Hook `PreToolUse` sur `Bash`. Intercepte les patterns destructifs avant exécution.

```
COMMAND = $( echo " $TOOL_INPUT " | jq
-r '.command' ) DANGEROUS = ( "rm -rf /"
"dd if=" "DROP DATABASE"
"git push --force.*main" ) for pattern in "
${DANGEROUS[@]} " ; do if [[ "
$COMMAND " = * "$pattern " * ]];
then echo "BLOCKED: $pattern " >& 2
exit 2 fi done exit 0
```

Exit code 2 bloque l'action et remonte le message stderr à Claude.

Git auto-stage après édition

Hook `PostToolUse` sur `Edit|Write`. Prépare automatiquement les fichiers modifiés pour le prochain commit.

```
INPUT = $( cat ) FILE = $( echo " $INPUT "
| jq -r '.tool_input.file_path // empty' )
[[ -z " $FILE " || " $FILE " =
"null" ]] && exit 0 git add " $FILE
" 2 > /dev/null exit 0
```

RTK auto-wrapper

Hook `PreToolUse` sur `Bash`. Détecte les commandes optimisables par RTK et informe Claude de l'alternative disponible.

La logique : si la commande commence par `git log`, `cargo test`, `pnpm list` ou des équivalents verbeux, suggérer `rtk <commande>` via stdout avant de laisser passer.

Logger toutes les commandes exécutées

Hook `PostToolUse` sur `Bash`. Utile pour l'audit et le debug de sessions longues.

```
COMMAND = $( cat | jq -r
'.tool_input.command // empty' ) echo " $( date
-u +%FT%T ) | $COMMAND " \
>>
~/claude/logs/commands.log exit 0
```

Pattern dispatcher : un seul point d'entrée

Plutôt que multiplier les entrées dans `settings.json`, un script `dispatch.sh` route vers des handlers spécialisés selon le fichier ou l'outil. Résultat : un seul matcher `Edit|Write|Bash` dans la config, et des handlers maintenables séparément dans `.claude/hooks/handlers/`.

Remplacement de sortie d'outil (v2.1.121)

Hook `PostToolUse`. Remplace ce que Claude reçoit comme résultat d'un outil via `hookSpecificOutput.updatedToolOutput`. Fonctionne pour tous les outils : Bash, Read, Write, Edit, MCP. Cas d'usage : anonymiser les données sensibles, compresser les résultats volumineux, injecter des métadonnées d'audit.

```
INPUT = $( cat ) OUTPUT = $( echo " $INPUT
" | jq -r '.tool_response.output // empty'
) # Anonymiser les tokens avant que Claude ne les voie
CLEAN = $( echo " $OUTPUT " | sed
's/ghp_[A-Za-z0-9]*/[TOKEN_REDACTED]/g' ) echo "{
\" hookSpecificOutput \" : { \" updatedToolOutput \"
: $( echo " $CLEAN " | jq -Rs . )
}" exit 0
```

Checklist installation

```
chmod +x .claude/hooks/mon-hook.sh # Obligatoire
# Tester manuellement avant d'activer : echo
'{"tool_name":"Edit","tool_input":{"file_path":"test.ts"}}'
\ | .claude/hooks/mon-hook.sh
```